CST2550 - Coursework 2

Detailed Pseudocodes and Analysis for Implemented Data Structures

Group Members		
Student ID	**Student Name/Surname**	**Role**
M01015301	Nunkoo Namrata-Riya	Scrum Master
M01038115	Hau Yam Seung Mélanie	Secretary
M01014266	Coret Mégane	Developer 1
M01026956	Ramtohul Syrielle	Developer 2
M01015305	Sedoyeka Rayan	Tester

Submission Date: 15/04/26
Lab Tutor: Dr. Gina Soopramanien

Table of Contents

1. Introduction 3
2. List 3
   2.1 Description 3
   2.2 Pseudocode 3
   2.3 Algorithm Analysis 8
3. Linked list 10
   3.1 Description 10
   3.2 Pseudocode 10
   3.3 Algorithm Analysis 13
4. Dictionary 13
   4.1 Description 13
   4.2 Pseudocode 13
   4.3 Algorithm Analysis 19

Table of Tables

Table 1: Time Complexity Analysis for List 9
Table 2: Time Complexity Analysis for Linked List 13
Table 3: Time Complexity Analysis of Dictionary 20

1. Introduction

Custom data structures for lists, linked lists and dictionaries have been implemented in the system. These are in the *Code/Data Structures folder* and are named **MyList**, **MyLinkedList** and **MyDictionary**.

This document outlines their implementation using pseudocode and provides a time complexity analysis of their operations.

2. List

2.1 Description

The list has been implemented using a fixed array, initially of size 10 – as values are being added the array is checked for availability of empty slots. If there is no vacant slot, the function Resize() is then called and a new array is created with double the size of the previous one and the data held in the previous array are copied into the new one.

2.2 Pseudocode

```
CLASS MyList:
    DECLARE items: ARRAY
    DECLARE capacity: INTEGER
    DECLARE count: INTEGER

    //procedure to initialise list to empty values
    PROCEDURE MyList()
        Capacity ← 10
        DECLARE ARRAY items OF SIZE capacity
        count ← 0
    END PROCEDURE

    //procedure to feed data to array (assigning an existing list)
    PROCEDURE MyList(IEnumerable<T> items)
        Capacity ← 10
        DECLARE ARRAY items OF SIZE capacity
        count ← 0
```

```
        //insert data into array
        FOR item ← 0 to LENGTH (items[]):
                CALL Add(item)
        NEXT
END PROCEDURE

//function for indexer as a getter
FUNCTION GetItem(index)
        //checking if index is out of bounds
        IF index < 0 OR index >= count THEN
                THROW EXCEPTION
        END IF
        RETURN items[index]
END FUNCTION

//procedure setItem as setter to assign value to array within range
PROCEDURE SetItem(index)
        IF index < 0 OR index >= count THEN
              THROW EXCEPTION
        END IF
        Items[index] ← value
END PROCEDURE

// function to get total number of elements in list
FUNCTION GetCount() AS INTEGER
        RETURN count
END FUNCTION
```

```
//function to check if list is empty
FUNCTION IsEmpty() AS BOOLEAN
        RETURN count == 0
END FUNCTION

//procedure to add one element to the list
PROCEDURE Add(value)
        //resize list if maximum capacity reached
        IF count = capacity THEN
                CALL Resize()
        END IF
        items[count] ← value
        count ← count + 1
END PROCEDURE

//manually increasing the size of the array to accommodate more items
PROCEDURE Resize()
        capacity ← capacity * 2
        CREATE  ARRAY resizedArr OF SIZE capacity
        FOR  index ← 0 TO count
                resizedArr [index] ← items[index]
        NEXT
        Items ← resizedArr
END PROCEDURE

//remove one element
FUNCTION Remove(value)
        FOR i ← 0 to _count
                IF items[i] == value THEN
```

```
                RemoveAt(i)
            ENDIF
        NEXT
END FUNCTION

//remove all elements from the list that satisfy the given condition (predicate)
PROCEDURE RemoveAll(match)
        FOR i ← 0 to _count
            IF match(items[i]) THEN
                CALL RemoveAt(i)
                i ← i - 1
            ENDIF
        NEXT
END PROCEDURE

//remove element at a specific index
PROCEDURE RemoveAt(index)
        IF index < 0 OR index >= count THEN
            THROW IndexOutOfRangeException()
        ENDIF
        // shift elements
        FOR i ← index TO (count -1)
            items ← items[i + 1]
        NEXT
        items[count - 1] ← default(T)
        count ← count - 1
END PROCEDURE
```

```
//reverse list
// as elements are fetched in descending order of date from the database, this will sort
// them in ascending order of date

PROCEDURE Reverse()

        DECLARE maxIndex : INTEGER

        DECLARE StartIndex : INTEGER

        maxIndex ← count – 1

        StartIndex ← 0

        WHILE StartIndex < maxIndex DO

                temp ← items[startIndex]

                items[StartIndex] ← items[maxIndex]

                items[maxIndex] ← temp

                StartIndex ← StartIndex + 1

                maxIndex ← maxIndex -1

        ENDWHILE

END PROCEDURE


// Insert match at correct position
// list is already in descending order of date, so this method will insert the new match at
// the correct position based on its scheduled date

PROCEDURE InsertMatch(newMatch)

        IF (count + 1) >= capacity THEN

                CALL Resize()

        ENDIF

        DECLARE itemDate: DATETIME

        itemDate ← newMatch.GameDate

         DECLARE i: INTEGER

        i ← count -1

        WHILE ( i >= 0 AND match == TRUE AND match.GameDate < itemDate) DO

                //right shift
```

```
            Items[ i + 1] ← Items[ i ]
            count ← count + 1
        ENDWHILE
        // insert new match at correct position
        Items[ i + 1] ← newMatch
        count ← count + 1
    END PROCEDURE

    //merge lists
    PROCEDURE AddRange(MyList<T> combiningList)
        FOR i ← 0 TO combiningList
            CALL Add(combiningList.items[i])
        NEXT
    END PROCEDURE

    //make lists iterable
    FUNCTION GetEnumerator()
        FOR i ← 0 TO count
            RETURN items[i]
        NEXT
    END FUNCTION
END CLASS
```

2.3 Algorithm Analysis

Process	Best case	Worst case	Reason
MyList()	O(1)	O(1)	
MyList(IEnumerable<T> items)	O(n)	O(n)	The constructor initialises the array using a FOR loop that iterates *n* times
GetItem(index)	O(1)	O(1)	Constant time due to direct array index access.
SetItem()	O(1)	O(1)	Constant time due to direct data assignment at a specific index.

GetCount()	O(1)	O(1)	Only returns a stored variable without any iteration
IsEmpty()	O(1)	O(1)	A single comparison operation is performed
Add (value)	O(1)	O(n)	Best case: space is available, so element is added directly Worst case: Maximum capacity is reached, so resizing is required which involves copying all elements into new array
Resize()	O(n)	O(n)	All elements must be copied into a new array, which iterates *n* times
Remove(value)	O(n)	O(n)	The algorithm needs to scan the array to find the element, and then shift the elements after removal, which takes linear times.
RemoveAll(match)	O(n)	$O(n^2)$	Best case: It must iterate through all *n* elements in the array, even when the condition is not matched. Worst case: If many elements match the condition, each removal requires shifting the array which leads to O(n) operations inside a O(n) loop
RemoveAt(index)	O(1)	O(n)	Best case: The last element is removed, so no shifting is required Worst case: Removing the first element when there is more than 1 element in the array, requiring all remaining elements to be shifted
Reverse()	O(n)	O(n)	The method performs a single pass through the array, swapping elements from both ends. Although half the elements are swapped, the constant factor is ignored.
InsertMatch(newMatch)	O(1)	O(n)	Best case: If the new element belongs to the end of the array, no shifting is required, and insertion is done directly. Worst case: If the new element, must be inserted at the beginning of the array, all existing elements must be shifted
AddRange(MyList<T> combiningList)	O(n)	O(n)	It iterates through each element in the new list and inserts it to the current list. As each Add() operation has an amortised time complexity of O(1), the overall time complexity is linear.
GetEnumerator()	O(1)	O(n)	It traverses through all *n* elements of array

Table 1: Time Complexity Analysis for List

3. Linked list

3.1 Description

The linked list has been implemented using objects created from the node class. One node consists of the data, and the previous and next node pointers.

In the *Node(value Procedure),* a node is instantiated with a value for the data, and its previous and next node pointers are set to NULL.

In the *AddFirst procedure*, a new node is added to the linked list, with its next pointer referencing the current head. The previous pointer of the current head is then updated to point to the new node. The new node is then assigned to the head variable. This allows elements to be added at the head of the linked list.

3.2 Pseudocode

```
CLASS MyLinkedList
    // class to represent each node in the linked list
    CLASS Node
        DECLARE value
        DECLARE next: Node
        DECLARE previous: Node
        PROCEDURE Node(value)
            next ← NULL
            previous ← NULL
            this.value ← value
        END PROCEDURE
    END CLASS

    DECLARE head: Node
    DECLARE count: INTEGER

    // initialise list
    PROCEDURE MyLinkedList()
        count ← 0
```

```
        END PROCEDURE

        // get number of elements
        FUNCTION GetCount() RETURNS INTEGER
                RETURN count
        END FUNCTION

        // get first element (the head)
        FUNCTION GetFirst() RETURNS Node
                RETURN head
        END FUNCTION

// add element at the head
FUNCTION AddFirst(value) RETURNS Node
        DECLARE newNode: Node
        newNode ← NEW Node(value)
        // check if the linked list has a head
        IF head != NULL THEN
                newNode.next ← head
                head.previous ← newNode
        END IF
        head ← newNode
        count ← count + 1
        RETURN newNode
END FUNCTION

// remove a node from linked list
FUNCTION Remove(node) RETURNS BOOLEAN
        IF count = 0 OR node = NULL THEN
```

```
            RETURN FALSE
        END IF
        IF node.previous != NULL THEN
            node.previous.next ← node.next
        ELSE
            // node is head
            head ← node.next
        END IF
        IF node.next != NULL THEN
            node.next.previous ← node.previous
        END IF
        count ← count - 1
        RETURN TRUE
END FUNCTION

// clear linked list
PROCEDURE Clear()
        head ← NULL
        count ← 0
END PROCEDURE

PROCEDURE Traverse()
        DECLARE node: Node
        node ← head
        WHILE current != NULL
            OUTPUT node.value
            node ← current.next
        END WHILE
END PROCEDURE
```

3.3 Algorithm Analysis

Process	Best case	Worst case	Reason
Node()	O(1)	O(1)	It only assigns values directly to variables
AddFirst(value)	O(1)	O(1)	It only performs comparisons and assigns values directly, without any search or traversal
Remove(node)	O(1)	O(1)	It only performs comparisons and reassigns values of the previous and next node, without any search or traversal
Clear()	O(1)	O(1)	It only reassigns values to variables
Traverse()	O(n)	O(n)	Performs *n* times iterations as each node in the linked list is visited
GetCount(), GetFirst()	O(1)	O(1)	Only one instruction (returning a stored value) is executed, therefore it is constant.

Table 2: Time Complexity Analysis for Linked List

4. Dictionary

4.1 Description

The dictionary has been implemented using a hash table that resolves collisions using separate chaining through a list. Each element is a hashNode consisting of the key, value and a pointer to the next element. A hash function maps each key to an index, and new entries are prepended to the chain at that index. When the load factor (capacity) reaches 0.75, the table resized by doubling capacity and rehashing all entries.

4.2 Pseudocode

```
CLASS MyDictionary
    CLASS HashNode
        DECLARE key
        DECLARE value
        DECLARE next: HashNode
        PROCEDURE HashNode(key, value)
            this.key ← key
            this.value ← value
            next ← NULL
        END PROCEDURE
```

```
END CLASS
DECLARE buckets: ARRAY OF HashNode
DECLARE size: INTEGER
CONSTANT capacity = 32
CONSTANT loadFactor = 0.75

// initialise dictionary
PROCEDURE MyDictionary()
    DECLARE ARRAY buckets OF SIZE capacity
    size ← 0
END PROCEDURE

// calculate and returns the index of bucket in hash table for a given key
FUNCTION GetBucketIndex(key) RETURNS INTEGER
    DECLARE hashCode: INTEGER
    hashCode ← HASH(key)
    RETURN ABS(hashCode MOD capacity)
END FUNCTION

// add element to hash table
PROCEDURE Add(key, value)
    IF key = NULL THEN
        THROW EXCEPTION
    END IF
    // check if size of hash table needs to be increased
    IF size >= capacity * loadFactor THEN
        CALL Resize()
    END IF
    DECLARE index: INTEGER
```

```
        index ← GetBucketIndex(key)
        DECLARE node: HashNode
        node ← buckets[index]
        WHILE node != NULL
                IF node.key = key THEN
                        node.value ← value
                        RETURN
                        END IF
                        node ← node.next
        END WHILE
        DECLARE newNode: HashNode
        newNode ← NEW HashNode(key, value)
        newNode.next ← buckets[index]
        buckets[index] ← newNode
        size ← size + 1
END PROCEDURE

// remove an element with the given key from hash table
FUNCTION Remove(key) RETURNS BOOLEAN
        IF key = NULL THEN
                THROW EXCEPTION
        END IF
        DECLARE index: INTEGER
        // find position of element in hash table
        index ← GetBucketIndex(key)
        DECLARE node: HashNode
        DECLARE prev: HashNode
        node ←   buckets[index]
        prev ← NULL
```

```
        WHILE node != NULL
            IF node.key = key THEN
                IF prev = NULL THEN
                    buckets[index] ← node.next
                ELSE
                    prev.next ← node.next
                END IF
                size ← size - 1
                RETURN TRUE
            END IF
            prev ← node
            node ← node.next
        END WHILE
        RETURN FALSE
END FUNCTION

// finds and returns element from the hash table with the given key
FUNCTION Get(key)
        IF key = NULL THEN
            THROW EXCEPTION
        END IF
        DECLARE index: INTEGER
        index ← GetBucketIndex(key)
        DECLARE node: HashNode
        node ← buckets[index]
        WHILE node != NULL
            IF node.key = key THEN
                RETURN node.value
            END IF
```

```
            node ← node.next
        END WHILE
        THROW EXCEPTION
END FUNCTION

//Checks if the hash table contains a specific key
FUNCTION ContainsKey(key) RETURNS BOOLEAN
        DECLARE index: INTEGER
        index ← GetBucketIndex(key)
        DECLARE node: HashNode
        node ← buckets[index]
        WHILE node != NULL
            IF node.key = key THEN
                RETURN TRUE
            END IF
            node ← node.next
        END WHILE
        RETURN FALSE
END FUNCTION

//attempts to retrieve the value associated with the given key
// returns true if key exists
FUNCTION TryGetValue(key) RETURNS BOOLEAN
        DECLARE index: INTEGER
        index ← GetBucketIndex(key)
        DECLARE node: HashNode
        node ← buckets[index]
        WHILE node != NULL
            IF node.key = key THEN
```

```
                    value ← node.value
                    RETURN TRUE
            END IF
            node ← node.next
        END WHILE
        value ← NULL
        RETURN FALSE
END FUNCTION

//resizing
PROCEDURE Resize()
        DECLARE newCapacity: INTEGER
        newCapacity ← capacity * 2
        DECLARE ARRAY newBuckets OF SIZE newCapacity
        FOR i ← 0 TO capacity - 1
            DECLARE node: HashNode
            node ←    buckets[i]
            WHILE node != NULL
                DECLARE nextNode: HashNode
                nextNode ← node.next
                DECLARE index: INTEGER
                index ← ABS(HASH(node.key) MOD newCapacity)
                node.next ← newBuckets[index]
                newBuckets[index] ← node
                node ← nextNode
            END WHILE
        NEXT
        buckets ← newBuckets
        capacity ← newCapacity
```

```
END PROCEDURE

//traversal algortithm
PROCEDURE Traverse()
    FOR i ← 0 TO capacity - 1
        DECLARE node: HashNode
        node ← buckets[i]
        WHILE node != NULL
            OUTPUT node.key, node.value
            node ← node.next
        END WHILE
    NEXT
END PROCEDURE
```

4.3 Algorithm Analysis

Process	Best case	Worst case	Reason
HashNode() global	O(1)	O(1)	All values are instantiated once therefore it is constant.
GetBucketIndex(key)	O(1)	O(1)	It only returns the hash code from the key passed to it, without any iterations or traversal
MyDictionary()	O(1)	O(1)	It only assigns values to variables
Add(key, value)	O(1)	O(n)	*Best case:* Resizing is not needed, and the slot calculated is free, the key-value pair is simply inserted or updated in the dictionary *Worst case:* Resizing or collision occurs, which require *n* iterations or full chain traversal through the *n* keys respectively
Remove(key)	O(1)	O(n)	Best case: The list of key-value pairs contains only the element to be removed, requiring only 1 pass through the WHILE loop Worst case: The list of key-value pairs contains *n* number of elements (> 1), requiring full chain traversal through the *n* keys
Get(key)	O(1)	O(n)	*Best case*: The list of key-value pairs contains only the element to be removed, requiring only

			1 pass through the WHILE loop to obtain the value *Worst case*: The list of key-value pairs contains *n* number of elements (> 1), requiring full chain traversal through the *n* keys to obtain the value
ContainsKey(key)	O(1)	O(n)	Similar to Get(key)
TryGetValue(key)	O(1)	O(n)	Similar to Get(key)
Resize()	O(n)	O(n)	All n existing nodes are rehashed into a new array that has twice the capacity of the previous one, which is a linear operation.
Traverse()	O(1) or O(n+k)	O(1) or O(n+k)	Iterates over all k slots (including empty ones) and all n stored nodes, where k is the current capacity.

Table 3: Time Complexity Analysis of Dictionary